

Midterm

Name: _____

Software Engineering — UATX — Spring 2026

Format	Pencil and paper. No computer, phone, notes, or coding agent.
Time	75 minutes (one class).
Length	Five questions on the actual exam, one from each section.
Coverage	Lectures 1.1 through 4.2 (git, files+SQL, REST/FastAPI, testing) plus assignments 1, 2, and 3.
Sections	Trace-through, spot the bug, specification, code review, short design.

HOW THIS EXAM THINKS ABOUT AGENTS

The whole semester has leaned into coding agents as the way real engineers work, so an exam that asks students to write working code from scratch on paper would punish them for taking the course seriously. Instead, every question is built around a skill the agent can't do for them: reading code carefully, finding bugs by inspection, writing the spec they'd hand to the agent, judging which of two implementations is better, and making design calls under uncertainty.

Question 1

Trace-through

This Python script runs to completion. Predict the value printed at A, B, and C. (.scalar() on a query result returns the single value from the first column of the first row — here, the count.)

```
from sqlalchemy import create_engine, text

engine = create_engine("sqlite:///memory:")

with engine.begin() as conn:
    conn.execute(text("""
        CREATE TABLE notes (id INTEGER PRIMARY KEY, body TEXT)
    """))
    conn.execute(text("INSERT INTO notes (body) VALUES (:b)"), {"b": "first"})
    conn.execute(text("INSERT INTO notes (body) VALUES (:b)"), {"b": "second"})

try:
    with engine.begin() as conn:
        conn.execute(
            text("INSERT INTO notes (body) VALUES (:b)"), {"b": "third"}
        )
        n = conn.execute(text("SELECT COUNT(*) FROM notes")).scalar()
        print("A:", n)                                # ____
        raise RuntimeError("oops")
except RuntimeError:
    pass

with engine.begin() as conn:
    n = conn.execute(text("SELECT COUNT(*) FROM notes")).scalar()
print("B:", n)                                         # ____

with engine.begin() as conn:
    conn.execute(text("UPDATE notes SET body = 'OOPS' WHERE id = 1"))
    conn.execute(text("INSERT INTO notes (body) VALUES (:b)"), {"b": "fourth"})

with engine.begin() as conn:
    rows = conn.execute(
        text("SELECT id, body FROM notes ORDER BY id")
    ).all()
print("C:", [(r.id, r.body) for r in rows])           # ____
```

For B, write one short sentence explaining *why*.

Question 2

Spot the bug

Your agent produced this test file for the BBS webserver from assignment 2. The tests pass on the first run.

```
# conftest.py
import pytest
from fastapi.testclient import TestClient
from main import app

@pytest.fixture
def client():
    return TestClient(app)

# test_users.py
def test_create_user_and_list_includes_them(client):
    r = client.post("/users", json={"username": "alice"})
    assert r.status_code == 201

    r = client.get("/users")
    users = r.json()
    assert len(users) >= 1
    assert any(u["username"] == "alice" for u in users)

def test_create_duplicate_username_returns_409(client):
    client.post("/users", json={"username": "bob"})
    r = client.post("/users", json={"username": "bob"})
    assert r.status_code == 409

def test_get_user_returns_user_data(client):
    r = client.get("/users/alice")
    assert r.status_code == 200
    assert r.json()["username"] == "alice"
```

The tests pass once. **Find at least three problems with this test file.** For each, one sentence on what's wrong, and one sentence on how you'd fix it.

Question 3

Specification

You're adding notification preferences to an app. Each user has settings for which events generate notifications, and via which channels. The known event types right now are: `new_message`, `new_follower`, `mentioned_in_comment`, `group_invite`. You expect the product team to ship roughly 5–10 new event types over the next year. Channels are stable and won't change: `email`, `push`, `in_app`. For every *(event, channel)* pair, the user can independently choose whether they want it. The default for new users is "everything on," with the one exception that `mentioned_in_comment` over `push` is off.

Write the schema spec. Cover all three:

1. **The shape decision.** You could put one column per *(event, channel)* directly on the `users` table — a "wide" approach — or keep preferences in a separate table. Pick one and in one sentence say why, naming one specific concern from the spec above that drives the call.
2. **The actual SQL** — `CREATE TABLE` statement(s) with column types, NOT NULL flags, defaults, and any keys.
3. **One ambiguity in the product spec above** that affects the data model and that you'd want to nail down before handing this off to your agent. State the ambiguity in one sentence, and the call you'd make in one sentence.

Question 4

Code review

Two test files for the checkout API of an e-commerce app. The product spec: adding an item to the cart returns 201; checkout charges payment, creates a paid `Order`, and empties the cart; checkout against an empty cart returns 400.

Version A:

```
def test_complete_checkout_flow(client, test_db):
    """Smoke test for the new-checkout release."""
    r = client.post("/api/cart/items", json={"sku": "X", "qty": 2})
    assert r.status_code == 201
    r = client.post("/api/cart/items", json={"sku": "Y", "qty": 1})
    assert r.status_code == 201

    cart = client.get("/api/cart").json()
    assert len(cart["items"]) == 2
    assert cart["total_cents"] == 4500

    r = client.post("/api/checkout", json={"payment_token": "tok_test"})
    assert r.status_code == 200
    order_id = r.json()["order_id"]

    cart = client.get("/api/cart").json()
    assert cart["items"] == [] # cart emptied by checkout

    order = client.get(f"/api/orders/{order_id}").json()
    assert order["status"] == "paid"
    assert order["total_cents"] == 4500
```

Version B:

```
def test_add_item_to_cart(client, test_db):
    r = client.post("/api/cart/items", json={"sku": "X", "qty": 2})
    assert r.status_code == 201
    assert r.json()["sku"] == "X"

def test_get_cart_lists_added_items(client, test_db):
    client.post("/api/cart/items", json={"sku": "X", "qty": 2})
    client.post("/api/cart/items", json={"sku": "Y", "qty": 1})
    items = client.get("/api/cart").json()["items"]
    assert len(items) == 2

def test_cart_total_sums_line_items(client, test_db):
    client.post("/api/cart/items", json={"sku": "X", "qty": 2}) # X is $15
    client.post("/api/cart/items", json={"sku": "Y", "qty": 1}) # Y is $15
    assert client.get("/api/cart").json()["total_cents"] == 4500
```

```
def test_checkout_creates_paid_order(client, test_db):
    client.post("/api/cart/items", json={"sku": "X", "qty": 2})
    r = client.post("/api/checkout", json={"payment_token": "tok_test"})
    assert r.status_code == 200
    order_id = r.json()["order_id"]
    assert client.get(f"/api/orders/{order_id}").json()["status"] == "paid"

def test_checkout_with_empty_cart_400s(client, test_db):
    r = client.post("/api/checkout", json={"payment_token": "tok_test"})
    assert r.status_code == 400
```

Pick one. Then:

1. Justify in three to five sentences. Mention at least two specific reasons.
2. Name *one specific behavior the version you DID NOT pick handles better*, in one sentence.
3. **What other tests would you add** to your picked version? Name two or three, one sentence each.

Question 5

Short design

You need to test this function:

```
import stripe
from sqlalchemy import text

class ChargeError(Exception): pass

def charge_user(user_id: int, amount_cents: int, db) -> str:
    """Charge the user's saved payment method via Stripe.
    Returns the Stripe charge id. Raises ChargeError on failure.
    Records an audit row in 'charges' on success."""
    row = db.execute(
        text("SELECT stripe_pm_id FROM payment_methods WHERE user_id = :u"),
        {"u": user_id},
    ).first()
    if row is None:
        raise ChargeError("no payment method on file")

    intent = stripe.PaymentIntent.create(
        amount=amount_cents,
        currency="usd",
        payment_method=row.stripe_pm_id,
        confirm=True,
    )
    if intent.status != "succeeded":
        raise ChargeError(f"stripe returned {intent.status}")

    db.execute(
        text("INSERT INTO charges (user_id, amount_cents, stripe_id) "
            "VALUES (:u, :a, :s)"),
        {"u": user_id, "a": amount_cents, "s": intent.id},
    )
    db.commit()
    return intent.id
```

Sketch the test plan. Cover all three:

1. **Boundaries.** For each of (a) the database and (b) the Stripe API, say whether your tests use the real thing, an in-memory or test version, or a mock — and why, in one sentence each.
2. **Three tests, one per behavior:** (i) the lookup of the payment method, (ii) the Stripe charge call, (iii) the audit-row insert. For each, give a `def test_*` name and 3–5 lines of pseudo-code showing the key assertion.
3. **One obvious-looking test you would NOT write**, with one sentence on why — something a less-careful tester would think they need but would actually be noise or a trap.

